

Code as Policies

arXiv:2209.07753

Code as Policies: Language Model Programs for Embodied Control Authors: Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, Andy Zeng Published: ICRA 2023

Abstract: Large language models have demonstrated the ability to generate code, but can this code be used to control robots? In this paper, we present Code as Policies (CaP), a framework that uses language models to generate code that controls robots. CaP extends the idea of chain-of-thought reasoning to robot control, allowing the model to generate executable code that interfaces with robot perception and action APIs.

Core Method: Code as Policies is a framework that uses LLMs to generate robot control programs in Python. The key insight is that code provides a more expressive and precise representation for robot policies than natural language.

The CaP Framework: 1. Natural Language Instruction: - User gives a task in natural language - Example: "Stack the red block on the blue block"

2. LLM Generates Code: - The LLM generates Python code that solves the task - Code uses provided perception and action APIs - The code can include logic, loops, conditionals - Hierarchical code generation: first generate high-level functions, then fill in details

3. Perception APIs: - Functions for detecting objects, getting positions - Example: `detect_objects()`, `get_position("red block")` - The LLM calls these to understand the environment

4. Action APIs: - Functions for robot movement and manipulation - Example: `move_to(position)`, `pick(object)`, `place(location)` - The generated code calls these to execute actions

5. Execution: - The generated code is executed on the robot - Perception results feed back into the code - The robot performs the task autonomously

Key Features: - Code is more expressive than natural language for policies - Supports complex logic: conditionals, loops, arithmetic - Hierarchical generation for complex tasks - Can generalize to new tasks by composing API calls - The LLM doesn't need robot-specific training

Why Code is Better than Natural Language for Policies: - Code has precise syntax and semantics - Code can express complex algorithms - Code is executable and verifiable - Code can use variables and state - Code supports composition and abstraction

Experiments: CaP was evaluated on robot manipulation tasks: - Successfully completed complex stacking and sorting tasks - Outperformed natural language policy baselines - Generalized to novel object configurations - Handled conditional tasks (e.g., "if red block exists, stack it") - Demonstrated arithmetic and spatial reasoning in code - The hierarchical approach improved success on long-horizon tasks

Key Findings: - Code generation is an effective interface for robot control - LLMs can generate meaningful robot programs with appropriate APIs - The approach enables zero-shot generalization to new tasks - Code provides better precision than natural language commands - Hierarchical code generation handles longer tasks

- CaP demonstrates the potential of LLM-generated code for embodied AI